pgmon 가이드 — PostgreSQL 텍스트 모니터링 툴 (설치 & 사용)

최종 수정: 2026-11 (pg_project 기준으로 갱신)

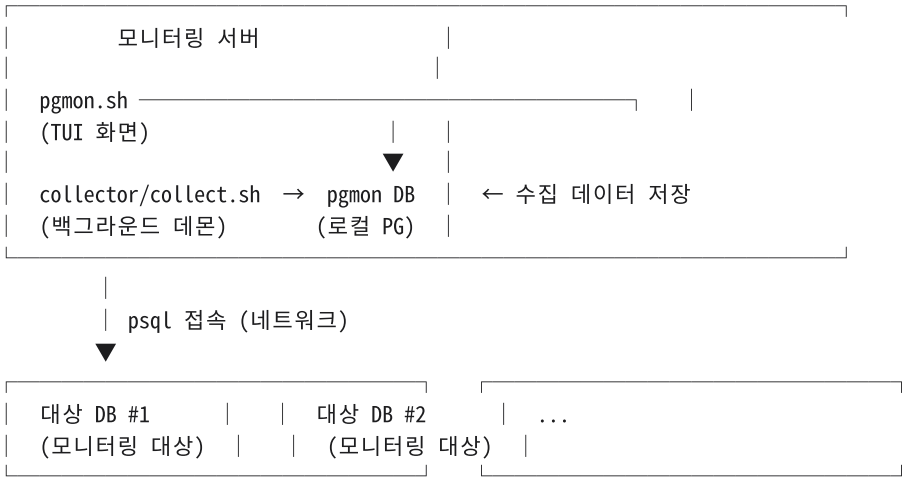
[목차]

1. 개요 및 구조
2. 사전 준비
   2-1. 모니터링 서버 OS 패키지
   2-2. 모니터링 저장소 DB (pgmon DB) 준비
   2-3. 대상 DB 준비
   2-4. 대상 DB 서버 OS 지표
3. 설치
4. 최초 실행 (초기화 절차)
5. 화면별 사용 가이드
   5-1. DB 선택 / 관리 화면
   5-2. 대시보드
   5-3. 세션 (Session)
   5-4. Statement
   5-5. Vacuum
   5-6. Object (테이블 / 인덱스)
   5-7. Alert
   5-8. 설정 (Settings)
   5-9. Report
   5-10. 복제 (Replication)
   5-11. Collector 상시 실행 / 알림
   5-12. 설치 / 수집 점검 (pgmon_check.sh)
6. 키 조작 요약
7. 파일 구조
8. 자주 발생하는 문제 (Troubleshooting)
9. 보안 주의사항

1. 개요 및 구조

pgmon은 PostgreSQL을 모니터링하는 터미널(TUI) 기반 셸 툴입니다.
별도의 웹 서버나 에이전트 없이 bash + psql 만으로 동작합니다.

[동작 방식]



- pgmon.sh : 화면 렌더링 / 사용자 입력 처리
- collect.sh : 백그라운드에서 메트릭 수집 → pgmon DB 저장
- pgmon DB : 수집 데이터 저장소 (모니터링 서버 로컬에 설치)
- 대상 DB : 실제 모니터링할 PostgreSQL 서버들

2. 사전 준비

2-1. 모니터링 서버 OS 패키지

pgmon은 아래 명령이 설치되어 있어야 합니다.

```
[ 필수 ]
  - bash 4.0 이상
  - psql (postgresql-client)   : DB 조회에 사용
  - bc                         : 소수점 계산 (CPU%, 비율 등)

[ 권장 ]
  - less                       : 긴 목록 페이저 (대부분 기본 설치)
  - tput (ncurses)             : 터미널 색상/커서 제어 (대부분 기본)

  ※ iostat(sysstat)은 필요 없습니다. CPU / MEM / Disk 지표는 모니터링 서버가
     아니라 대상 DB 서버의 /proc 을 읽어서 계산합니다 (2-4 참고).

[ RHEL / Rocky 설치 명령 ]
  sudo dnf install -y bc
  # psql 은 postgresql_install.sh 로 설치한 엔진의 bin 사용 (~/.postgresrc)

[ 설치 확인 ]
  psql --version
  bc --version
```

2-2. 모니터링 저장소 DB 준비 (pgmon DB)

pgmon은 수집 데이터를 저장하기 위해 PostgreSQL DB를 하나 사용합니다.
모니터링 서버 자체에 PostgreSQL을 설치하고 초기화합니다.

▶ PostgreSQL 설치 (모니터링 서버에 없는 경우)

패키지에 포함된 설치 스크립트로 single 모드 설치:

```
bash postgresql_install.sh    → 1) single 선택
```

▶ pg_hba.conf — 로컬 접속 허용 확인

pgmon 스크립트는 127.0.0.1 (또는 localhost) 로 접속합니다.
pg_hba.conf 에 아래 항목이 있는지 확인하세요.

```
# TYPE  DATABASE  USER     ADDRESS         METHOD
host    all       all      127.0.0.1/32    md5
# 또는 scram-sha-256
```

파일 위치: $PGDATA/pg_hba.conf (기본 /pgdata/data/pg_hba.conf)
수정 후 반드시 reload:

```
pg_ctl -D $PGDATA reload
```

▶ init.sql 동작

init.sql 은 psql 의 \gexec 로 pgmon 데이터베이스를 생성합니다.
dblink 등 별도 extension 은 필요 없습니다.
DB 이름은 pgmon 으로 고정이므로 "저장소 DB명" 은 pgmon 그대로 두세요.

▶ 비밀번호 기본값 안내

init.sql 이 자동으로 생성하는 역할(role)과 기본 비밀번호:

역할명	기본 비밀번호	용도
pgmon_owner	pgmon_owner	스키마 소유자

```
  pgmon_writer     pgmon_writer          수집 데이터 쓰기 (pgmon.sh 사용)
  pgmon_reader     pgmon_reader          읽기 전용

※ 운영 환경에서는 초기화 후 반드시 비밀번호를 변경하세요.
   ALTER ROLE pgmon_writer PASSWORD '새비밀번호';

※ pgmon 최초 실행 시 "저장소 DB 유저"는 pgmon_writer 로 입력하고,
   비밀번호는 위 기본값(또는 변경한 값)을 입력합니다.
```

2-3. 대상 DB 준비 (모니터링할 PostgreSQL 서버)

▶ 접속 계정 준비

```
pgmon이 대상 DB에 접속할 전용 모니터링 계정을 만드세요.
pg_monitor 역할을 부여하면 대부분의 시스템 뷰를 조회할 수 있습니다.

  CREATE ROLE pgmon_monitor WITH LOGIN PASSWORD 'pgmon_monitor';
  GRANT pg_monitor TO pgmon_monitor;
  -- pg_terminate_backend (Kill 기능) 사용 시 superuser 또는
  -- 해당 세션과 동일 유저여야 합니다.

pg_monitor 역할이 없는 구버전 (PG 9.x 이하) 은 아래와 같이:
  GRANT SELECT ON pg_stat_activity TO pgmon_monitor;
  GRANT SELECT ON pg_stat_replication TO pgmon_monitor;
  GRANT SELECT ON pg_stat_statements TO pgmon_monitor;
  -- 등 필요한 뷰에 개별 GRANT
```

▶ pg_hba.conf — 모니터링 서버에서의 접속 허용

```
대상 DB 서버의 pg_hba.conf 에 모니터링 서버 IP를 허용 추가:

  host    all    pgmon_monitor    <모니터링서버IP>/32    md5

수정 후: pg_ctl -D $PGDATA reload   (HA 는 patronictl reload)
```

▶ pg_stat_statements extension 설치 (Statement 화면 필수)

```
※ pg_project 의 postgresql_install.sh 로 설치한 DB 는 shared_preload_libraries
   등록 + CREATE EXTENSION 까지 기본 적용돼 있어 이 단계가 필요 없습니다.
   (그 외 방법으로 설치한 DB 만 아래 절차 수행)

대상 DB의 postgresql.conf:
  shared_preload_libraries = 'pg_stat_statements'

설정 후 PostgreSQL 재시작:
  pg_ctl -D $PGDATA restart   (HA 는 patronictl restart <클러스터명>)

이후 대상 DB에서:
  CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

※ pgmon 최초 실행 시 extension 설치 자동 시도를 수행합니다.
   단, shared_preload_libraries 설정은 수동으로 해야 합니다.
```

2-4. 대상 DB 서버 OS 지표 (CPU / MEM / SWAP / Disk I/O)

```
OS 지표는 대상 DB 서버 기준입니다. 대상 서버의 /proc 을 1초 간격 2회 읽어
계산하며, 아래 순서로 시도합니다.

  1) DB 접속 (pg_read_file) — 모니터링 계정에 아래 권한 필요
       GRANT pg_read_server_files TO pgmon_monitor;
       GRANT EXECUTE ON FUNCTION pg_read_file(text) TO pgmon_monitor;
```

```
    ※ DB 등록 시 [1] 자동생성 을 고르면 자동 부여됩니다.
    ※ pg_read_server_files 는 DB 서버 파일 읽기 권한입니다.
       보안 정책상 곤란하면 REVOKE 하고 2) SSH 방식을 사용하세요.

  2) SSH — 1) 이 안 될 때만 사용
       모니터링 서버 → 대상 DB 서버 SSH 키 교환 필요 (패스워드 입력 안 함)
         ssh-keygen -t rsa -N ""
         ssh-copy-id -p <포트> <SSH유저>@<대상DB>
       SSH 유저/포트는 DB 등록 시 입력 (기본: 현재 유저 / 22)

  3) 둘 다 실패 → 대시보드에 "수집 불가" 표시, collector 로그에 기록

  대시보드 Host Resources 제목 옆에 (DB 서버 · pg_read_file) /
  (DB 서버 · SSH) 로 어떤 방식으로 가져왔는지 표시됩니다.

▶ (선택) pg_buffercache / pgstattuple

  pgmon은 대상 DB 등록 시 아래 extension도 자동으로 설치 시도합니다.
  설치 실패해도 해당 기능만 비활성화되고 나머지는 정상 동작합니다.

    pg_buffercache  : 버퍼 캐시 히트율 (Dashboard)
    pgstattuple     : 정밀 Bloat 계산 (Object > Bloat 뷰)

  사전에 superuser로 미리 설치하려면:
    CREATE EXTENSION IF NOT EXISTS pg_buffercache;
    CREATE EXTENSION IF NOT EXISTS pgstattuple;
```

3. 설치

```
▶ 압축 해제

  tar -xzf pgmon.tar.gz
  cd pgmon

▶ 실행 권한 확인

  chmod +x pgmon.sh
  chmod +x collector/collect.sh
  chmod +x lib/*.sh

▶ 디렉토리 구조 확인

  pgmon/
  ├── pgmon.sh              # 메인 실행 파일
  ├── lib/
  │   ├── ui.sh             # 화면 공통 함수
  │   ├── init.sh           # 초기화 / DB 등록 관리
  │   ├── dashboard.sh      # 대시보드
  │   ├── session.sh        # 세션
  │   ├── statement.sh      # pg_stat_statements
  │   ├── vacuum.sh         # Vacuum
  │   ├── object.sh         # 테이블 / 인덱스
  │   ├── alert.sh          # Alert
  │   ├── settings.sh       # 수집 주기 설정
  │   └── report.sh         # 리포트
  ├── sql/
  │   └── init.sql          # 저장소 DB 초기화 SQL
  ├── collector/
  │   └── collect.sh        # 백그라운드 수집 데몬
  └── conf/                 # 실행 중 자동 생성
      ├── pgmon.conf        # DB 접속 정보 저장
      └── *.log             # collector 로그
```

4. 최초 실행 (초기화 절차)

```
cd pgmon
bash pgmon.sh
```

conf/pgmon.conf 파일이 없으면 최초 실행으로 인식하고
아래 순서로 초기화를 진행합니다.

[Step 1] 저장소 DB 정보 입력

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 저장소 DB 호스트  [127.0.0.1]  : 엔터 (기본값 사용)    │
│ 저장소 DB 포트    [5432]       : 엔터                  │
│ 저장소 DB명       [pgmon]      : 엔터                  │
│ 저장소 DB 유저    [pgmon_writer]: 엔터                 │
│ 저장소 DB 패스워드             : pgmon_writer 입력     │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

※ 저장소 DB는 "2-2. 모니터링 저장소 DB 준비" 를 완료한 후 입력

[Step 2] Superuser 정보 입력 (init.sql 실행 용, 일회성)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ Superuser 유저명  [postgres]   : 엔터                  │
│ Superuser 패스워드             : postgres 비밀번호 입력  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

→ init.sql 이 실행되어 role / database / schema / table 생성
→ 완료 후 Superuser 정보는 어디에도 저장되지 않습니다.

[Step 3] 대상 DB 수 입력
등록할 DB 수 [1] : 모니터링할 DB 개수 입력

[Step 4] 대상 DB 정보 입력 (등록 수만큼 반복)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 닉네임           : 예) prod-db-1                       │
│ 호스트 / IP      : 예) 192.168.1.10                    │
│ 포트             [5432]                                │
│ DB명             : 예) myapp                           │
│ 접속 유저        : 예) pgmon_monitor                   │
│ 패스워드         : 비밀번호 입력                        │
│ 데이터 보관 주기 [15] : 일 단위, 기본 15일             │
│ OS 지표용 SSH 유저 [현재유저] : pg_read_file 불가 시 사용 │
│ OS 지표용 SSH 포트 [22]                                │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

→ 입력 완료 시 자동으로 연결 테스트 수행
→ 연결 성공 시 모니터링 role 설정 선택
  [1] 자동생성 : superuser 로 pgmon_monitor 생성 + 권한 부여
                 (pg_monitor, pg_read_all_stats, pg_read_server_files 등)
  [2] 기존유저사용 : 권한 확인 후 부족한 GRANT 안내
  [s] 건너뜀
→ 연결 성공 시 extension 자동 설치 시도

[Step 5] 완료
→ conf/pgmon.conf 저장 (권한 600, 비밀번호 포함되므로 주의)
→ DB 선택 화면으로 진입

[이후 실행]
conf/pgmon.conf 가 있으면 바로 DB 선택 화면으로 진입합니다.

5. 화면별 사용 가이드

5-1. DB 선택 / 관리 화면

pgmon 시작 후 가장 먼저 나오는 화면입니다.
등록된 DB 목록과 현재 연결 상태(● 연결됨 / ✘ 연결 불가)를 보여줍니다.

```
  1)  prod-db-1    192.168.1.10:5432   ● 연결됨
  2)  prod-db-2    192.168.1.11:5432   ✘ 연결 불가
  m)  DB 관리 (추가/수정/삭제)
  q)  pgmon 종료

[ DB 관리 (m 선택) ]
  a) 신규 DB 추가        : 새로운 대상 DB 등록
  e) 기존 DB 정보 수정   : 호스트/포트/비밀번호 등 수정
  d) DB 삭제             : 등록 해제 (수집 데이터는 DB에 보존)
```

5-2. 대시보드

대상 DB를 선택하면 기능 메뉴가 나옵니다. 1번 선택.

자동 갱신 주기: 기본 30초 (settings에서 변경 가능)

```
[ 표시 항목 ]
  Server Info
    - DB 버전, 호스트명/IP, DB명, 가동 시간
    - DB 상태: active(primary) / standby(recovery)
    - HA: Standalone / PRIMARY (replica 목록, lag) / STANDBY (replay lag)

  Host Resources
    - CPU 사용률   : 게이지 바 + % (70% WARN, 90% CRIT 색상 변경)
    - Memory       : 게이지 바 + MB (75% WARN, 90% CRIT)
    - Swap         : 게이지 바 + MB (50% WARN, 80% CRIT)
    - Disk I/O     : Read/Write KB/s

  Connections
    - Total / Active / Idle / Idle in Transaction / Long(>5m)
    - Long SQL(>30s), WAL Size

  Locks
    - Blocked 세션 수
    - 상위 3개 blocking chain (blocking PID → blocked PID, 경과 시간)

  Maintenance
    - Vacuum 실행 중 수, Dead tuples 총합, Top bloat 테이블

  Status
    - 전체 판정: OK / WARN / CRIT 배지
    - 미해결 Alert 목록 (CRIT=빨강, WARN=노랑)

[ 조작 키 ]
  r      : 즉시 갱신
  q      : 기능 메뉴로
  ESC    : DB 선택 화면으로
```

5-3. 세션 (Session)

현재 접속 세션을 실시간으로 조회합니다.
기본 갱신 주기: 15초

```
[ 상단 요약 ]
  Total / Active / Idle / Idle-in-Tx / Long(>5m) / Blocked

[ 세션 목록 ]
  PID / User / App / State / Wait Event / Duration / Query (80자)
```

```
  색상 의미:
    초록  : active 세션
    노랑  : idle in transaction 세션
    빨강  : Lock 대기(blocked) 세션

[ 조작 키 ]
  f  : 필터 순환
       all → active → idle → idle_tx → long → blocked → all
  s  : 정렬 순환
       dur(실행시간) → pid → user → state → dur
  k  : 특정 PID 강제 종료 (pg_terminate_backend)
       입력 예) 12345
       ※ 권한 있는 계정이어야 종료 가능
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로

-----------------------------------------------------------------
```

5-4. Statement

```
pg_stat_statements 기반 SQL 성능 분석 화면입니다.
pg_stat_statements extension이 설치되지 않은 경우 설치 안내 메시지 표시.

[ 컬럼 ]
  # / Calls / Mean(ms) / Total(ms) / Max(ms) / Cache Hit% / Query

  색상 의미:
    Mean(ms): 1000ms 이상 노랑, 5000ms 이상 빨강
    Cache Hit%: 90% 미만 노랑, 70% 미만 빨강

[ 조작 키 ]
  s  : 정렬 순환
       mean(평균시간) → total(합계시간) → calls(호출수) → rows → cache → mean
  d  : 상세보기 — 번호 입력 → 전체 SQL + 통계 (calls / mean / max / rows /
       cache hit / temp blks / 실행 유저)
  t  : 튜닝 — 번호 입력 → pg_tune 의 pgtune tune 실행 결과 표시
       · 파라미터($1, $2 ...)가 있으면 실제 값을 넣은 SQL 을 한 줄로 입력
       · 실행 계정 선택 (모니터링 계정은 업무 테이블 조회 권한이 없을 수 있음)
       · 기본은 EXPLAIN 만 (쿼리 실행 안 함), y 입력 시 EXPLAIN ANALYZE
       · 인덱스 후보 검증은 대상 DB 에 hypopg 필요 (없으면 CREATE EXTENSION 할지 물어봄)
       · 실행계획 대안(Hash/Merge Join, 인덱스 강제) 실험은 pg_hint_plan +
         superuser 계정일 때만
       · pg_tune 위치: PGTUNE_CMD → ../../tools/pg_tune → ../pg_tune → PATH
       · 로그: pgmon/conf/pgtune_log/
  r  : pg_stat_statements 통계 초기화 (pg_stat_statements_reset())
       확인 프롬프트 후 실행
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로

-----------------------------------------------------------------
```

5-5. Vacuum

```
Vacuum / Autovacuum 현황과 Wraparound 위험도를 확인합니다.

[ 뷰 구성 ]
  tables      : 테이블별 dead tuple 비율, 최근 autovacuum 시각
  wraparound  : XID 나이와 Wraparound 위험도 %

[ 상단 공통 ]
  현재 실행 중인 Vacuum / Autovacuum 프로세스 목록 (PID, 경과시간, 쿼리)

[ tables 뷰 컬럼 ]
  Table / Live / Dead / Dead Ratio / Last Autovac / AutoVac cnt
  Dead Ratio: 20% 이상 노랑, 50% 이상 빨강

[ wraparound 뷰 컬럼 ]
```

```
  Table / Age(XIDs) / MaxAge / Risk%
  Risk%: 50% 이상 노랑, 80% 이상 빨강

[ 조작 키 ]
  v  : 뷰 전환 (tables ↔ wraparound)
  m  : 수동 VACUUM ANALYZE 실행
       테이블명 입력 예) public.orders
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로
```

5-6. Object (테이블 / 인덱스)

테이블과 인덱스의 크기, 접근 패턴, Bloat를 분석합니다.

```
[ 뷰 구성 ]
  t (table)      : 테이블 크기 및 Seq/Idx 스캔 비율, Dead tuples
  i (index)      : 인덱스 크기, 스캔 횟수, unique/primary 여부
  u (unused_idx) : 미사용 인덱스 목록 (scan=0, 1MB 이상, PK/UK 제외)
  b (bloat)      : pgstattuple 기반 정밀 Bloat 분석 (상위 10개 테이블)
                   ※ pgstattuple extension 필요, 수 초 소요될 수 있음

[ table 뷰 정렬 (s 키) ]
  size(전체크기) → seq(Seq Scan 많은 순) → dead(Dead Tup 많은 순)

[ 조작 키 ]
  t / i / u / b : 뷰 전환
  s  : 정렬 변경 (table 뷰에서만 적용)
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로
```

5-7. Alert

Alert 발생 현황과 임계값을 관리합니다.

```
[ 뷰 구성 ]
  a (active)  : 미해결 Alert 목록 (WARN=노랑, CRIT=빨강)
  h (history) : Alert 발생/해소 이력 최근 100건
  c (config)  : 임계값 설정 목록 (global / db-specific 구분)

[ 기본 임계값 (init.sql에서 설정됨) ]
  Metric              WARN   CRIT
  ──────────────────────────────────────────────────────────────

  cpu_usage_pct         70     90
  mem_usage_pct         75     90
  swap_usage_pct        50     80
  lock_count             5     20
  idle_in_tx_sessions    5     15
  long_sessions          3     10
  dead_ratio_pct        20     50
  replication_lag_sec   30    120

  ※ collector가 30초마다 자동으로 체크하고 기록합니다.
  ※ 조건 해소 시 resolved_at이 자동으로 업데이트됩니다.

[ 조작 키 ]
  a / h / c : 뷰 전환
  r  : Alert 수동 해소처리
       ID 입력 또는 "all" 입력으로 전체 해소
  e  : 임계값 수정
       Alert ID 입력 → 새 WARN / CRIT / 활성화 여부 입력
       (엔터만 치면 기존값 유지)
  n  : DB 전용 임계값 추가
       글로벌 설정 외에 특정 DB에만 적용되는 임계값 추가
```

```
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로
```

5-8. 설정 (Settings)

메트릭별 수집 주기와 데이터 보관 주기를 설정합니다.

```
[ 표시 항목 ]
  - 메트릭 수집 주기 (ID / Metric / Interval(sec) / Enabled)
  - 데이터 보관 주기 (일)
  - Collector 실행 상태 (실행 중 PID 또는 중지 여부)

[ 기본 수집 주기 ]
  Metric       기본(초)
  ──────────────────────────────────────
  dashboard       30
  session         15
  statement       60
  vacuum         120
  object         300
  lock            15

[ 조작 키 ]
  e  : 수집 주기 수정
       Config ID 입력 → 새 초(sec) 입력 → 활성화 여부 입력
  r  : 데이터 보관 주기 변경 (일 단위)
  t  : Collector 시작 / 중지 토글
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로
```

5-9. Report

수집된 데이터를 기간별로 집계해서 보여줍니다.
화면을 실시간으로 갱신하지 않으므로 대화형으로 분석하기 좋습니다.

```
[ 기간 선택 (p 키) ]
  1h / 6h / 24h / 7d / 30d
  보관 주기(기본 15일) 이내의 데이터만 조회됩니다.

[ 뷰 선택 (v 키) ]
  summary    : 전체 요약 + Dead Tuples 추이 스파크라인
  cpu        : CPU / Memory / Swap 시간대별 집계
  session    : 세션 수 시간대별 집계
  lock       : Lock 발생 이력
  vacuum     : 테이블별 Dead Ratio 최대값 집계
  repl       : 복제 지연(초 / bytes) + 슬롯 WAL 적체 시간대별 최대값
  growth     : 테이블 크기 증가 — 기간 첫 수집 대비 마지막 수집 (Top 30)
  statement  : SQL 평균 실행시간 집계 Top 20
  alert      : Alert 발생 건수 시간대별 집계

[ CSV 내보내기 (x 키) ]
  현재 보고 있는 뷰를 CSV로 저장합니다. (summary / cpu 제외)
  저장 위치: pgmon/conf/reports/pgmon_{닉네임}_{뷰}_{기간}_{일시}.csv

[ 조작 키 ]
  p  : 기간 변경
  v  : 뷰 변경
  x  : CSV 내보내기
  q  : 기능 메뉴로
  ESC: DB 선택 화면으로
```

5-10. 복제 (Replication)

기능 메뉴 9번. 대상 DB 를 실시간 조회합니다 (10초 자동 갱신).

```
[ Primary 인 경우 ]
  연결된 standby 별: Client / App / State / Sync / Lag(bytes) / Lag(sec) / Replay LSN
  standby 가 하나도 없으면 경고 표시

[ Standby 인 경우 ]
  WAL receiver 상태(streaming 여부) / upstream / receive·replay LSN /
  미적용 WAL 크기 / replay 지연(초) / 마지막 replay 트랜잭션 시각

[ 공통 ]
  복제 슬롯: active 여부 / WAL 적체량 / wal_status
    ※ inactive 슬롯은 WAL 을 계속 붙잡아 디스크가 찰 수 있음 (노랑)
  최근 1시간 지연 추이 (collector 수집분, 5분 단위 최대값)

[ 관련 Alert ]
  replication_lag_sec     : 기본 WARN 30초 / CRIT 120초
  repl_slot_retained_mb   : 기본 WARN 1024MB / CRIT 10240MB (메시지에 슬롯명 표시)
  dead_ratio_pct          : 기본 WARN 20% / CRIT 50% (dead 1,000건 이상 테이블, 메시지에 테이블명)

[ 기존 설치본 ]
  저장소 DB 에 init.sql 을 한 번 다시 실행해야 수집됩니다 (몇 번 돌려도 안전):
    psql -U postgres -d postgres -f pgmon/sql/init.sql
```

5-11. Collector 상시 실행 / 알림

collector 는 pgmon 화면을 종료해도 계속 수집한다 (일반 계정 프로세스, root 불필요).

```
  bash pgmon_ctl.sh status              상태 확인
  bash pgmon_ctl.sh start [닉네임|all]
  bash pgmon_ctl.sh stop  [닉네임|all]  (watchdog 재기동 대상에서도 제외)
  bash pgmon_ctl.sh cron-install        재부팅 자동 시작 + 5분 watchdog 을 사용자 crontab 에 등록
  bash pgmon_ctl.sh log <닉네임>

  ※ 상시 수집을 하려면 설치 후 cron-install 을 한 번 실행해 두세요.

[ 알림 ]
  cp conf/notify.conf.sample conf/notify.conf  후 채널 설정
    log     : conf/alerts.log (기본)
    command : 메일 / 사내 메신저 스크립트 실행 (내용은 PGMON_* 환경변수)
    webhook : JSON POST (Slack / Teams 형식)
  변경 후: bash pgmon_ctl.sh restart
```

5-12. 설치 / 수집 정상 여부 점검 — pgmon_check.sh

등록을 마친 뒤 한 번 실행해서 PASS / FAIL 리포트를 받는다.

```
    bash pgmon_check.sh                # C1~C6
    bash pgmon_check.sh --alert-test   # C7 Alert 발생 시험 포함 (1~2분 추가)
    bash pgmon_check.sh --wait 40      # C5 대기 시간 (기본 70초)

  C1  저장소 DB    접속 / pgmon 스키마 / 쓰기 권한
  C2  대상 DB      접속 / Primary·Standby / pg_monitor / pg_stat_statements
                   / OS 지표 경로 (pg_read_file 또는 SSH)
  C3  collector    실행 / 상시(watchdog 대상) / crontab
  C4  수집 최신성  메트릭별 마지막 수집 시각 vs 주기
  C5  수집 증가    N초 동안 스냅샷이 실제로 늘었는지
  C6  수집 값      CPU / MEM 값, HA 역할이 실제 DB 와 맞는지
  C7  Alert        임시 임계값(lock_count WARN 0)으로 발생 확인 → 바로 원복
```

결과: conf/pgmon_check_<시각>.txt

6. 키 조작 요약

키	동작
q	현재 화면에서 한 단계 뒤로 (기능 메뉴 → DB 선택)
ESC	DB 선택 화면으로 바로 이동
Ctrl+C	pgmon 화면 종료 (collector 는 계속 수집 — 끄려면 pgmon_ctl.sh stop)
r	즉시 갱신 (대시보드)
f	필터 순환 (세션)
s	정렬 변경 (세션 / Statement / Object)
k	PID Kill (세션)
v	뷰 전환 (Vacuum / Alert / Report)
m	수동 VACUUM 실행 (Vacuum)
p	기간 변경 (Report)
x	CSV 내보내기 (Report)
e	항목 수정 (Settings / Alert)
t	Collector 토글 (Settings) / pg_tune 튜닝 (Statement)
d	SQL 상세보기 (Statement)

7. 파일 구조

```
pgmon/
 ├─── pgmon.sh                  메인 실행 파일
 ├─── lib/
 │    ├─── ui.sh                 색상 / 헤더 / 게이지 등 공통 UI 함수
 │    ├─── init.sh               초기화 / conf 저장 / DB 등록 관리
 │    ├─── dashboard.sh          대시보드 화면
 │    ├─── session.sh            세션 화면
 │    ├─── statement.sh          Statement 화면
 │    ├─── vacuum.sh             Vacuum 화면
 │    ├─── object.sh             Object 화면
 │    ├─── alert.sh              Alert 화면
 │    ├─── settings.sh           설정 화면
 │    └─── report.sh             Report 화면
 ├─── sql/
 │    └─── init.sql              저장소 DB 초기화 SQL
 │                              (role / database / schema / table / index 생성)
 ├─── collector/
 │    └─── collect.sh            백그라운드 수집 데몬
 │                              (pgmon.sh 에서 DB 선택 시 자동 실행)
 └─── conf/                      실행 중 자동 생성
      ├─── pgmon.conf            DB 접속 정보 (권한 600)
      ├─── .collector_1.pid      collector 프로세스 PID
      ├─── collector_{닉네임}.log collector 실행 로그
      └─── reports/              CSV 내보내기 저장 폴더
```

8. 자주 발생하는 문제 (Troubleshooting)

[문제 1] "필수 명령이 없습니다: psql bc"
  → OS 패키지 설치 항목(2-1) 참고
  → psql 이 PATH 에 없으면: source ~/.postgresrc

[문제 2] 저장소 DB 초기화 실패
  → superuser 유저/패스워드 확인 (init.sql 은 superuser 로 실행)
  → 저장소 DB 호스트의 pg_hba.conf 에서 127.0.0.1 접속 허용 확인

[문제 3] 연결 테스트 실패 — "연결 실패"
  → 대상 DB의 pg_hba.conf 에 모니터링 서버 IP 추가 여부 확인
  → 방화벽에서 5432 포트 허용 여부 확인
  → 입력한 유저/비밀번호/DB명 확인

[문제 4] Statement 화면에 "pg_stat_statements extension 이 설치되지 않았습니다"
  → 대상 DB의 postgresql.conf 에 아래 설정 추가 후 재시작:
      shared_preload_libraries = 'pg_stat_statements'
  → 재시작 후 대상 DB에서:
      CREATE EXTENSION pg_stat_statements;

[문제 5] Host Resources 가 "수집 불가" / CPU·MEM 이 0 으로 표시됨
  → 대상 DB 서버 /proc 을 읽지 못한 상태 (2-4 참고)
  → DB 방식: GRANT pg_read_server_files, pg_read_file(text) EXECUTE 확인
  → SSH 방식: ssh -o BatchMode=yes <SSH유저>@<대상DB> cat /proc/loadavg 로 확인

[문제 6] 화면이 깨져 보임 (색상 문자 오염, 줄 정렬 어긋남)
  → 터미널 창 너비 80 이상 권장 (120 이상 최적)
  → TERM 환경변수 확인: echo $TERM (xterm-256color 권장)
  → PuTTY 사용 시 Window > Translation: UTF-8 설정 필요

[문제 7] Collector가 "중지됨" 으로 표시됨
  → Settings 화면에서 t 키로 재시작
  → 로그 확인: cat conf/collector_{닉네임}.log | tail -30
  → pgmon.conf 의 REPO_PASS 가 변경되었다면 conf 파일 직접 수정 후 재시작

[문제 8] Report 화면에 데이터가 없음
  → Collector가 실행 중인지 Settings 에서 확인
  → 최초 실행 후 데이터가 쌓이기까지 수집 주기만큼 시간이 필요
  → dashboard 기본 수집 주기 30초이므로 30초 후 재확인

[문제 9] Object > Bloat 뷰에 "pgstattuple extension 필요" 메시지
  → 대상 DB에서: CREATE EXTENSION IF NOT EXISTS pgstattuple;
  → superuser 또는 pg_monitor 권한 필요

9. 보안 주의사항

① conf/pgmon.conf 에는 DB 비밀번호가 평문으로 저장됩니다.
   파일 권한은 자동으로 600(소유자만 읽기)으로 설정되지만,
   pgmon 실행 계정을 전용 OS 계정으로 분리하는 것을 권장합니다.

② init.sql 에 설정된 기본 비밀번호를 반드시 변경하세요.
   ALTER ROLE pgmon_owner PASSWORD '새비밀번호';
   ALTER ROLE pgmon_writer PASSWORD '새비밀번호';
   ALTER ROLE pgmon_reader PASSWORD '새비밀번호';
   → pgmon.conf 의 REPO_PASS 도 동일하게 수정해야 합니다.

③ 대상 DB 접속 계정(pgmon_monitor)은 최소 권한만 부여하세요.
   pg_monitor 역할이면 충분하며, superuser는 사용하지 마세요.

④ pg_terminate_backend (세션 Kill) 기능은 권한이 있는 경우에만
   동작합니다. 실수로 서비스 세션을 종료하지 않도록 주의하세요.

⑤ conf/reports/ 폴더의 CSV 파일에 SQL 및 쿼리 텍스트가 포함될 수
   있습니다. 외부 반출 시 민감 정보 포함 여부를 확인하세요.

END OF GUIDE